

Google Compute Engine (GCE)

Virtual Machines on Google's Global Infrastructure

GCE lets you run virtual machines in the cloud with full control of the OS, networking, and applications.

WHAT IS COMPUTE ENGINE?

Compute Engine is Google Cloud's **Infrastructure as a Service (IaaS)** that provides scalable virtual machines on Google's global infrastructure.

- Full control of OS and applications
- Scalable and reliable infrastructure
- Global network with high performance
- Pay only for what you use

HOW IT WORKS – UNDER THE HOOD

1. You create a VM

2. Google reserves resources

3. VM is provisioned on physical servers

4. Your OS boots and is ready to use

Inside a Google Data Center

Physical Server

VM 1
(Your Linux Server)

VM 2

VM 3

VM 4

...

Hypervisor (KVM)

Compute (CPU)

Memory (RAM)

Local SSD / Storage

Network (Google Backbone)

You Get

vCPUs
 Memory
 Persistent Disk
 Internal & External IP

MACHINE TYPES (Examples)

Family	Best For	Example Machine Type	vCPU	Memory
E2 (General Purpose)	Cost-effective Dev/Test, Small workloads	e2-medium	2	4 GB
N2 (General Purpose)	Enterprise apps, Web apps, Databases	n2-standard-4	4	16 GB
C2 (Compute Optimized)	High CPU Performance Applications	c2-standard-8	8	32 GB
M2 / M3 (Memory Optimized)	Memory-intensive workloads, SAP HANA	m2-standard-8	8	64 GB+

STORAGE – PERSISTENT DISK

VM
 Persistent Disk
 Google Distributed Storage
Replicated across multiple devices in multiple locations

Durable

Highly Available

Survives Hardware Failures

NETWORKING

Every VM runs in a VPC (Virtual Private Cloud)

VPC Network

Zone A
VM
VM

Zone B
VM
VM

Zone C
VM
VM

Internal IP: 10.10.1.5

Optional IP: 34.95.x.x

Traffic between VMs in the same VPC uses Google's private global network (not the public internet).

FIREWALL

Firewall rules are applied at the VPC level.

Example Rule:

Allow: TCP 22 (SSH)

Source: 10.0.0.0/8

=> SSH allowed only from internal network 10.0.0.0/8

AUTO SCALING WITH MANAGED INSTANCE GROUPS

Load Balancer
 Managed Instance Group (MIG)

Auto Scaling

CPU > 70%
↑ Add more VMs

CPU < 20%
↓ Remove VMs

✓ Automatically creates VMs ✓ Replaces unhealthy VMs ✓ Scales based on load ✓ Ensures high availability

HIGH AVAILABILITY

Deploy across multiple zones or regions for resilience.

Zone A

VM
VM

Traffic

Zone B

VM
VM

If one zone fails, your application keeps running.

SECURITY – SERVICE ACCOUNTS

VM

VMs have an identity. They can securely access Google Cloud services without storing passwords or keys.

Example: BigQuery

Uses short-lived tokens automatically. More secure. No secrets on the server.

WHEN TO USE (AND NOT USE) COMPUTE ENGINE

Use Compute Engine when you need:

- ✓ Full OS control and root access
- ✓ Legacy or lift-and-shift applications
- ✓ Custom software / libraries
- ✓ Specialized networking needs
- ✓ Stateful workloads (databases, SAP, Oracle, custom apps)

Consider Alternatives when you need:

- ✗ Simple web apps or APIs
- ✗ Containerized applications
- ✗ Event-driven workloads
- ✗ Want to reduce server management

Cloud Run
 GKE
 App Engine

KEY BENEFITS

- Global Infrastructure**
Deploy in 40+ regions and 100+ zones
- High Performance Network**
Google's premium backbone network
- Scalable**
Scale up or out in minutes
- Secure by Design**
IAM, VPC, Firewall, Shielded VMs
- Cost Effective**
Sustained use discounts, custom machine types, committed use

Cloud Maturity Journey

On-Premises Servers
 Compute Engine
 Containers (GKE)
 Serverless (Cloud Run / App Engine)

One Sentence Summary

Google Compute Engine is GCP's IaaS service that provides secure, scalable virtual machines with full control of the operating system, running on Google's global infrastructure.

Google Compute Engine (GCE) – The Real-World Explanation

Most beginners learn:

"Compute Engine is Google's Virtual Machine service."

That's technically correct, but it doesn't explain why Compute Engine exists, when to use it, or how it actually works in production.

What Compute Engine Really Is

At its core, Compute Engine is a service that rents you a portion of Google's global infrastructure.

When you create a VM, Google is not creating a physical server.

Instead:

1. Google has massive data centers worldwide.
2. Each data center contains thousands of physical servers.
3. A hypervisor (Google uses KVM-based virtualization) partitions those servers.
4. Your VM is allocated CPU, RAM, disk, and network resources from those physical machines.

Conceptually:

```
Physical Server
|
├── VM 1 (Your Linux Server)
├── VM 2
├── VM 3
└── VM 4
```

You see a server.

Google sees a virtualized workload running on shared infrastructure.

Why Compute Engine Exists

Many workloads need complete control over the operating system.

Examples:

- Legacy applications
- Oracle databases
- SAP workloads
- Custom networking software
- Third-party software requiring root access
- Applications migrated from on-premises

These workloads cannot simply be deployed to:

- Cloud Run
- App Engine
- Cloud Functions

because they need full OS access.

That's where Compute Engine comes in.

What You Actually Get

When you create a VM, Google gives you:

CPU

Virtual CPUs (vCPUs)

Example:

```
4 vCPU
16 GB RAM
```

Your application gets processing power.

Memory

RAM allocated to the VM.

Example:

```
8 GB
16 GB
32 GB
128 GB
```

Used for:

- Application runtime
- Caching
- Database buffers

Storage

Usually attached through Persistent Disk.

```
VM
|
└ Persistent Disk
```

The VM can be deleted while the disk survives.

This is a huge difference from traditional servers.

Networking

Each VM receives:

```
Internal IP
External IP (optional)
```

Example:

```
10.10.1.5
34.95.x.x
```

The VM becomes a first-class citizen on Google's global network.

Understanding Machine Types

A machine type is simply a bundle of CPU and RAM.

Example:

```
e2-medium
```

Contains:

```
2 vCPU
4 GB RAM
```

Common Families

E2

General purpose.

Cheap
Flexible
Dev/Test
Small workloads

Most cost-effective.

N2

Performance-oriented.

Enterprise applications
Java applications
Databases

C2

Compute optimized.

High CPU
Low latency

Used for:

- Trading systems
- Rendering
- Scientific workloads

M Series

Memory optimized.

Example:

M2
M3

Used for:

- SAP HANA
- Large databases

Memory can reach terabytes.

Boot Process of a VM

When you click "Create Instance":

Google performs something similar to:

1. Reserve CPU
2. Reserve RAM
3. Attach Disk
4. Configure Network
5. Start Hypervisor
6. Boot OS

The OS boots exactly like a physical machine.

Linux kernel starts.

Services start.

Application starts.

Persistent Disk Architecture

Many engineers think:

VM owns Disk

Wrong.

Actually:

```
graph TD
  VM --> PD[Persistent Disk]
  PD --> DSS[Distributed Storage System]
```

Google stores disk blocks across multiple devices.

Benefits:

- Durable
- Replicated
- High availability

If a physical disk dies:

Application keeps running

because the storage backend is distributed.

Networking in Production

Every VM lives inside a VPC.

```
graph TD
  VPC --> VM1
  VPC --> VM2
  VPC --> VM3
```

Communication happens through Google's backbone network.

Not through the public internet.

Benefits:

- Lower latency
- Better security
- Higher throughput

Firewall Reality

Firewall rules are applied at the VPC level.

Example:

```
Allow:
TCP 22

Source:
10.0.0.0/8
```

Meaning:

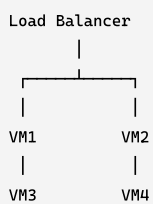
```
SSH allowed only internally
```

This is far more scalable than managing OS-level firewalls on thousands of machines.

Auto Scaling

One VM is rarely enough.

Production architecture:



Google uses:

Managed Instance Groups (MIG)

MIG automatically:

- Creates VMs
- Removes VMs
- Replaces failed VMs
- Scales based on load

Example:

```
CPU > 70%
```

Add more servers.

```
CPU < 20%
```

Remove servers.

High Availability

Single VM:

```
BAD
```

If it fails:

```
Application down
```

Production:

Zone A

└─ VM1

└─ VM2

Zone B

└─ VM3

└─ VM4

Now a zone failure doesn't take down the application.

Security Model

A VM has an identity.

This surprises many engineers.

Example:

VM

|

└─ Service Account

The VM can authenticate to GCP services without passwords.

Example:

VM

↓

BigQuery

Google automatically provides short-lived tokens.

No secrets stored on the server.

This is one of the most important security practices in GCP.

When Experienced Engineers Choose Compute Engine

Use Compute Engine when you need:

Full OS Control

Install anything
Configure anything
Root access

Legacy Applications

Oracle
SAP
Custom Java Apps
Windows Servers

Lift-and-Shift Migrations

Move:

On-Prem VM
↓
Compute Engine VM

with minimal changes.

Stateful Applications

Applications that need:

Custom storage
Custom networking
Specific OS tuning

When NOT to Use Compute Engine

Many teams overuse VMs.

If you're simply running:

REST API
Python App
NodeJS App
Container

consider:

- Cloud Run
- GKE
- App Engine

These reduce operational burden significantly.

A common cloud maturity journey is:

On-Prem
↓
Compute Engine
↓
Containers
↓
Cloud Run / Serverless

The One-Sentence Summary

Google Compute Engine is GCP's Infrastructure-as-a-Service offering that gives you virtual machines running on Google's global infrastructure, providing full control over the operating system, networking, storage, and runtime environment while Google manages the underlying physical hardware and virtualization layer.